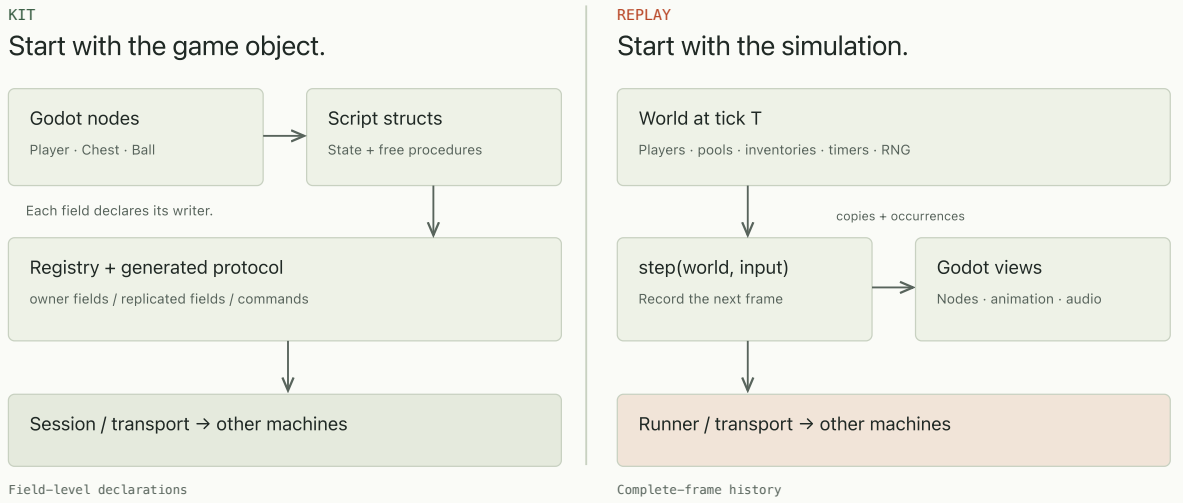
ENGINEERING / MULTIPLAYER SYSTEMS

A world on every machine.

Making a game with Odin and Godot means deciding where state lives, who can change it, and what happens when two machines disagree.

Kit & Replay / Illustrated guide + working model / Source 44654014

FIG. 01 TWO AUTHORING MODELS



The useful distinction is the unit of state: fields on registered entities, or a complete frame that can be restored and stepped again.

Suppose two players reach the same chest. Both press Take. Each screen should respond immediately, but the item must end up in one inventory. The game needs a rule for choosing the winner and a way to repair the losing prediction.

This repository has two independent ways to write that game. **Kit** works from script structs, tagged fields, and generated commands. **Replay** works from a typed world, recorded inputs, and a step function. The distinction matters most when an action changes several things at once.

Both frameworks can run co-op games. Both have an authoritative prediction path. Kit's cooperative examples are a convenient starting point; Replay makes the complete simulation history explicit. We will follow the same pickup through each design.

BUILT WITH ODIN × GODOT / PLAYABLE EXAMPLE

Take it into the yard.

Scrapyard is an arena shooter built with Odin and Godot. Play the browser build on itch.io to try the movement and combat, then follow the code through a complete game.



Captured from the live browser build on itch.io.

Play in your browser ↗ Study the Replay Scrapyard port →

The repository's Replay port has a visual contributor guide and a file-by-file code guide covering input, World, session startup, and Godot views.

---

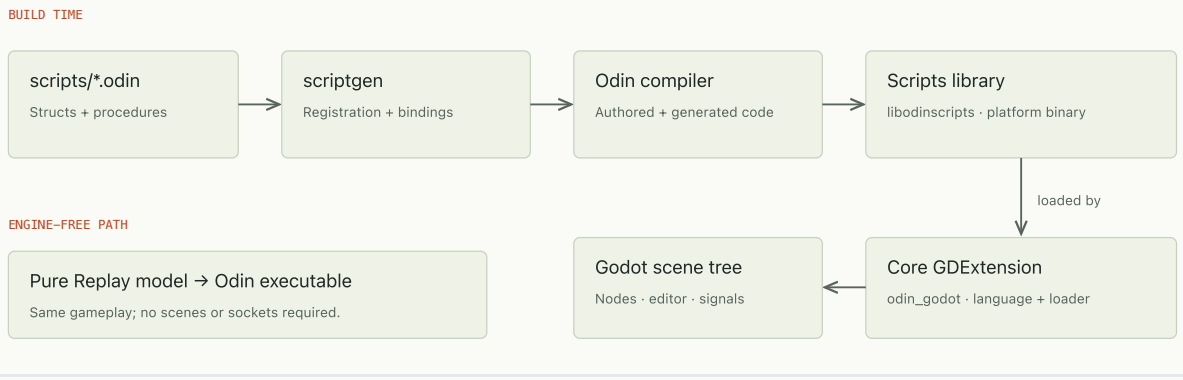
01 / THE ENGINE BOUNDARY

Godot is still the editor.

You still author scenes, place nodes, wire signals, and use the Inspector. An Odin script is a struct plus free procedures, attached to an engine node. The first field, conventionally `owner`, holds that node's handle. Your `self` pointer addresses the script data; `self.owner` addresses the engine object.

The build runs `scriptgen` over the scripts, generates registration and bindings, then compiles the scripts library. The core GDExtension registers Odin with Godot and loads that library. Kit and Replay sit on this boundary in different ways.

FIG. 02 FROM SOURCE TO A RUNNING SCENE



The core extension and the scripts library are separate. Pure Replay models also run in an ordinary Odin executable.

Read the implementation guide: project wiring · scripts and structs

---

02 / KIT

Start with the object. Declare its writers.

In Kit, the network registry knows each entity's type, ID, and owning player. Field tags decide how its data travels. A host-owned field such as health uses `gd:"replicate"`. Cooperative motion can use `gd:"owner"`: the owner writes its pose, while other machines interpolate received samples.

The small player in `examples/hello_net` makes this split visible:

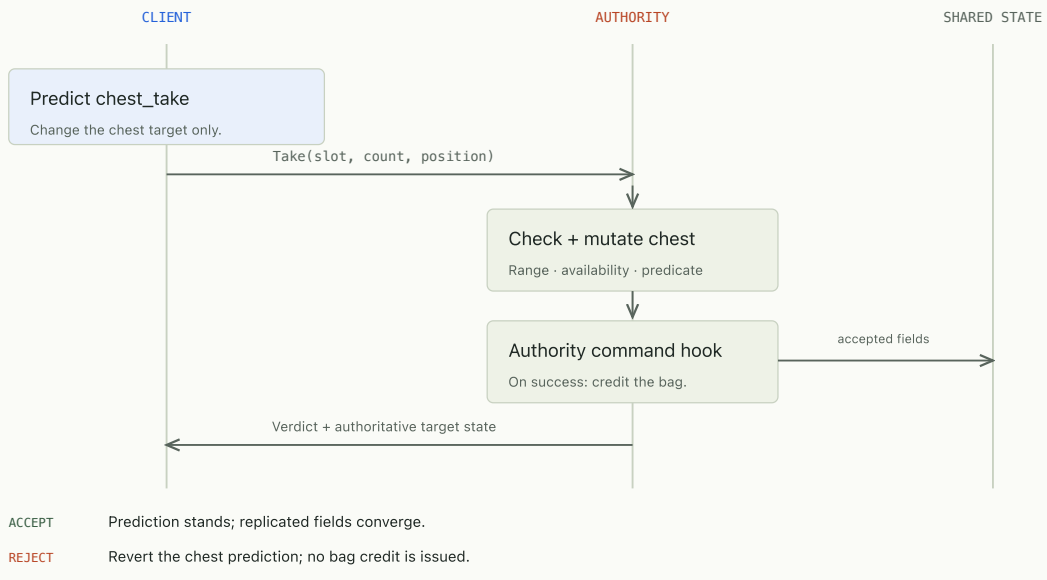
examples/hello_net/scripts/player.odin · excerpt

```
Player :: struct {
    owner:  gd.Node2d,
    skin:   gd.Polygon2d `gd:"onready=Skin"`,
    net_id: knet.Net_Id,
    x, y:   f32 `gd:"owner,wire=f16"`,
    pid:    u8  `gd:"replicate"`,
    mine:   bool,
    tinted: bool,
}
```

Taking an item is a command. A `@(gd_command)` procedure checks the request and mutates its target. The generated wrapper handles submission, optional local prediction, and the host's verdict. The host's accepted changes travel through ordinary field replication.

There is an important boundary: **a predicted Kit command mutates only its target**. The chest command removes the item from the chest. Crediting the player's bag belongs in the authority command hook, after acceptance. Code that changes the chest, bag, quest, and enemy state inside the predicted command crosses that boundary.

FIG. 03 A KIT PICKUP



The target prediction and the cross-entity consequence have different owners. The authority hook is the place to connect them.

> KIT ALSO PREDICTS
>
> For server-driven motion, `kit/sim` uses `gd:"predict,interp"` fields and a pure `@(gd_tick)` procedure. The Hello Sim example promotes the cooperative square to this lane. Choosing Kit does not commit the game to trusting client movement.

Command rules · Kit prediction tutorial

---

03 / REPLAY

Make the whole tick restorable.

Now give the pickup more consequences. It empties the chest, changes a bag, destroys an entity, and perhaps advances a quest. If a correction restores only the chest, the remaining values describe an event that no longer happened.

Replay puts mutable gameplay dependencies in one `World`. A frame contains that state at a tick. The runner saves frames and inputs; a step transforms one frame into the next. Cross-entity changes are ordinary writes within that world.

```
frame at T  +  input for T+1  →  frame at T+1  +  occurrences
```

---

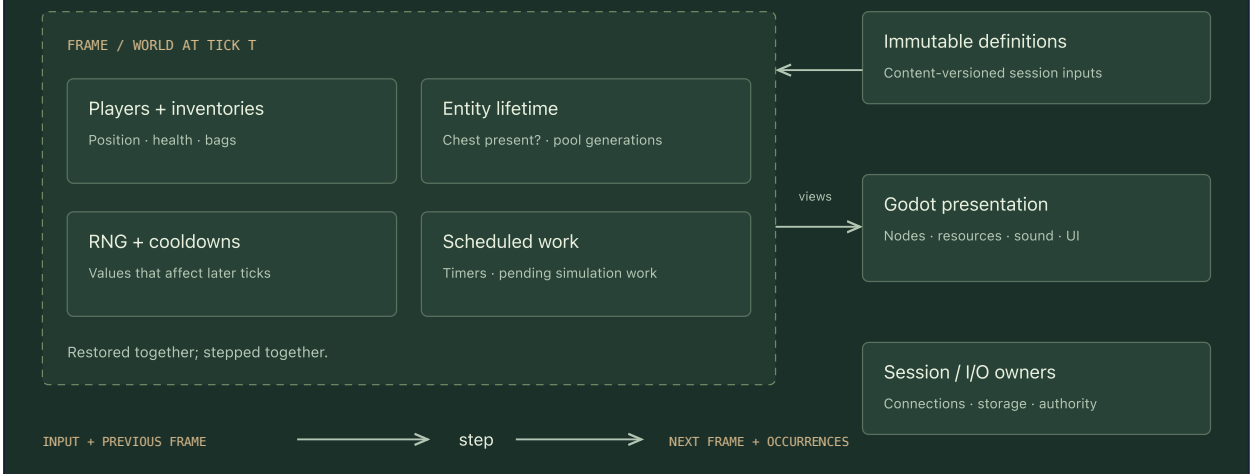
examples/replay_loot/game/game.odin · excerpt

```
World :: struct {
    players:  [2]Player,
    entities: r.Pool(Entity, 16),
    chest:    r.Handle(Entity),
    rng:      r.Rng,
}

// Inside step, after movement and the range check:
amount := chest.items
p.bag += amount
r.pool_remove(&w.entities, w.chest)
r.emit(ctx, u64(i + 1), u64(control.seq) << 8 | 1,
       Event{.Item_Taken, u8(i), p.x, amount})
```

The frame includes more than visible positions. RNG, cooldowns, pending work, entity existence, and pool allocation metadata affect the next tick. They need to be restored too. Node references, sockets, textures, and live audio stay outside it.

FIG. 04 WHAT BELONGS IN A FRAME?



Restore the values needed to compute the next tick. Present the result through detached views.

A replayed step must read its state, typed input, tick, and fixed definitions. Reading the wall clock or a live Godot transform introduces a dependency that the recorded frame cannot reproduce. Playing a sound inside the step repeats a side effect whenever the runner replays it.

Instead, `emit` records an occurrence with an identity: `(epoch, tick, source, key)`. The presentation journal compares old and replacement occurrences and reports Begin, Update,

Confirm, or Cancel. Effects are attached to those changes outside the simulation.

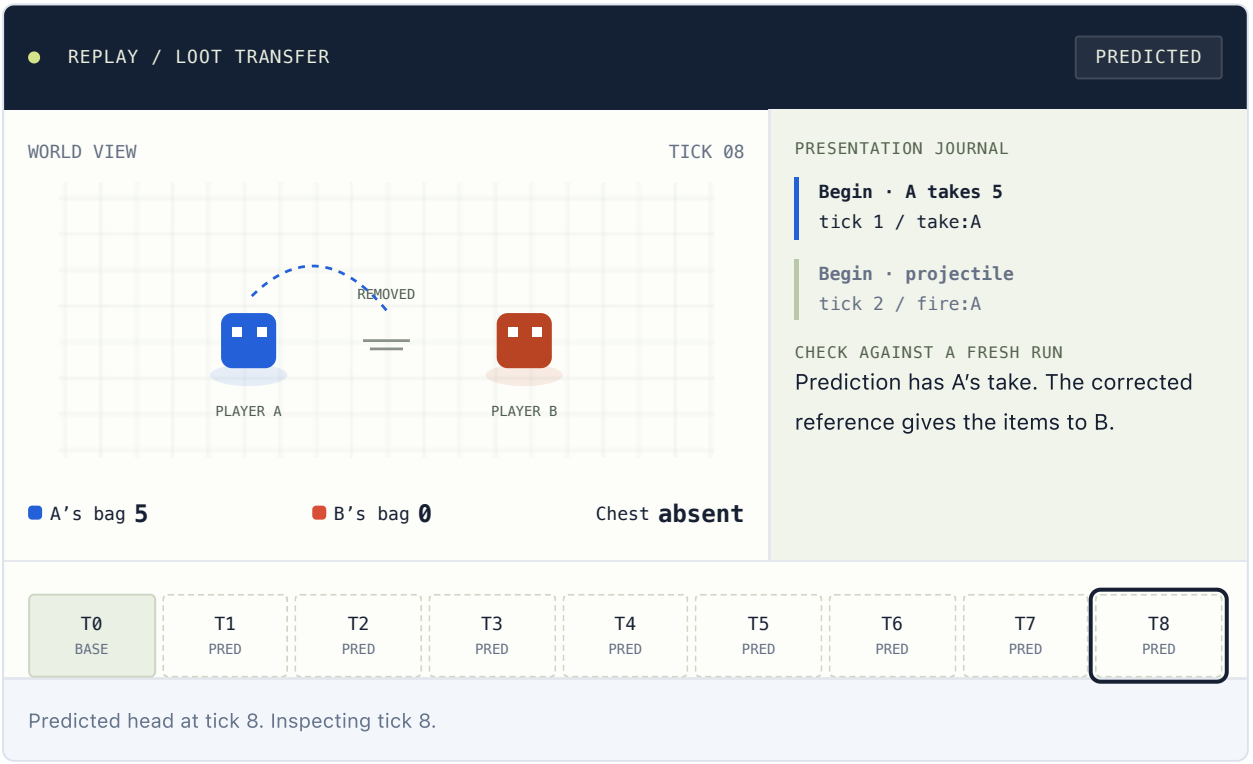
Frame and transaction contract · Presentation journal · Complete loot model

04 / WORKING MODEL

Change the winner. Rebuild the consequences.

The local Loot executable predicts that player A takes five items at tick 1. At tick 8, it revises that input: player B took them. The runner restores tick 0 and rebuilds ticks 1–8. An independent run with the corrected input checks the result.

The model below isolates that operation. Apply the correction, then replay one tick at a time. During replay, the view previews staged frames and the pending journal diff; the live revision would publish only after the complete suffix succeeds.



Browser illustration, not the Odin runtime or a network benchmark. It models a pickup at tick 1 and an unrelated projectile at ticks 2–4; the full example also restores pool allocation and RNG.

In this local experiment, `revise_input` replaces a recorded input batch. In an authoritative network game, the runner instead verifies the authority's frame and calls `correct`, then replays the speculative suffix. These operations share the restore-and-replay idea, but an input revision does not establish network authority.

The complete suffix is staged before publication. If any step fails, the revision does not leave half of the world corrected. This protects frame state; it cannot undo a sound, file write, or global mutation performed inside a callback.

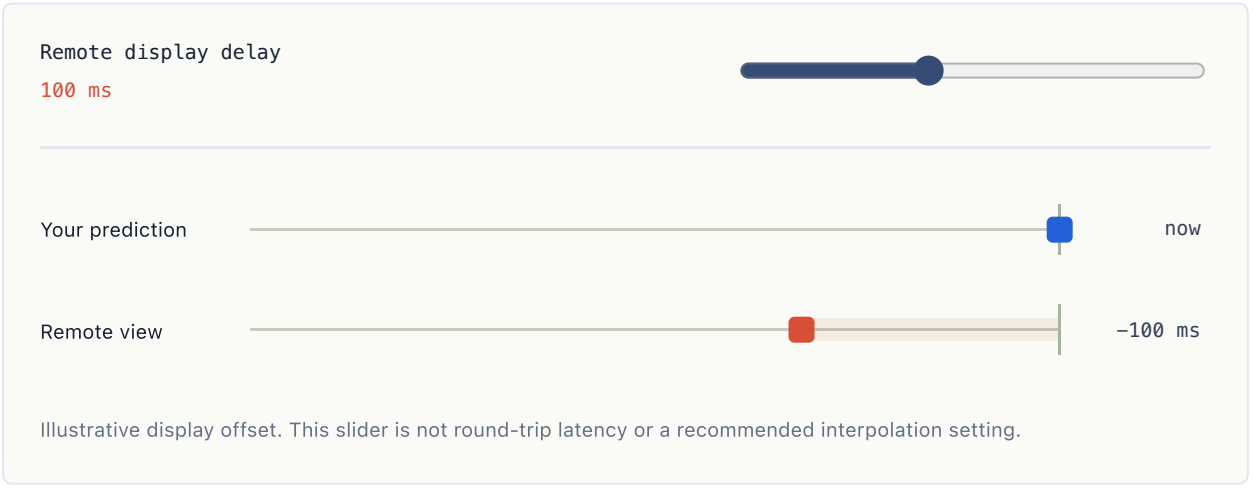
---

05 / PRESENTATION

One screen can show two different moments.

Your own avatar responds to current input. A remote avatar usually comes from a short buffer of received history. Drawing both at the same nominal "now" would require predicting the remote player's next decision.

This matters for effects. If a remote player picks up a gem, removing it as soon as the fact arrives can make it vanish before the visible avatar reaches it. The watched effect needs to follow the watched display clock.



Illustrative display offset. This slider is not round-trip latency or a recommended interpolation setting.

A contested ball adds another choice. Kit's Claimball predicts the local player and the ball, while watching the opponent in the past. Its presentation claim weights the ball toward the local

predicted pose while the local player is driving it. When that influence ends, the ball blends back toward the watched view. The claim changes presentation; the authority still resolves the game.

Godot rigid-body physics needs a separate policy. Saving a node's pose does not save the solver's contact history. Replay's owner-stream profile records accepted body observations. It does not turn the Godot contact solver into a deterministic rollback simulation.

Kit timelines · Replay owner streams

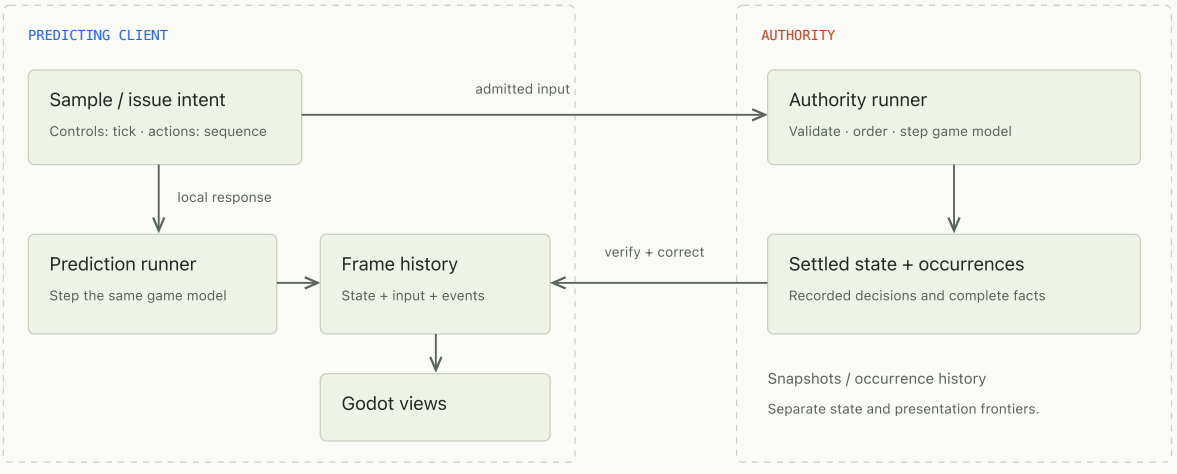
---

06 / NETWORK POLICY

Choose who decides.

A framework does not choose the game's trust model. A listen host is a player's process; a dedicated authority is infrastructure. Deterministic peers exchange inputs under a different contract. Use the smallest example that matches the policy you intend to ship.

FIG. 05 REPLAY / AUTHORITATIVE NETWORK PATH



Server-authoritative policy. Cooperative body streams and deterministic peers use different contracts.

Controls carry ticks. Actions carry sequence IDs and an authored delivery policy. Authority state corrects the prediction; occurrence history also has to reach presentation.

TRUSTED PLAYERS / OWNER-DRIVEN MOTION

Let the player drive the body; let the host settle shared state.

KIT

Start with hello_net. Owner fields carry movement; host fields carry shared state. Commands resolve interactions such as taking loot or transferring control.

REPLAY

Use Party → Room for the invitation and recovery lifecycle. Use owned body observations when cooperative motion depends on live Godot physics.

An owner's motion observations are trusted under this policy. A migration successor needs the complete authority backup and a valid authority transition, not just its visible client state.

`What changes when you choose a framework`

Question	Kit	Replay
Where do I write gameplay?	Script structs, generated commands, optional sim ticks.	A typed World with step/action callbacks, outside the scene tree.
What is restored?	Command target state; sim-lane predicted history under its contract.	A complete frame for the authored simulation group.
How do I change several entities?	Keep command prediction target-local; use the authority hook for consequences.	Mutate the frame's entities in the same simulation operation.
How does Godot draw it?	Generated presentation fields and per-frame script code.	Copied views and identity-aware occurrence changes.
What should I read first?	Kit quickstart, then Hello Sim.	Choosing a runner, then Loot and Hello.

07 / BUILD THE FIRST VERSION

A small game with a useful failure case.

Build one room, two players, and a chest with one item. Add movement first, then make both players take the item under injected delay. This gives prediction, authority, and correction something observable to do before you add combat or procedural content.

1

Get a square on screen.

Use the existing scene and project wiring. In Kit, start with `examples/hello_net`. In Replay, start with the offline Hello scene. Keep input sampling and drawing easy to find.

2

Give the item a single owner.

Kit: put the chest mutation in a command and bag credit in its authority hook. Replay: put both bags, the chest, and its lifetime in World. Define how simultaneous requests are ordered.

3

Force the prediction to lose.

Delay the connection or revise the local Loot input. Check that the losing inventory is repaired and its pickup effect is canceled. Repeating the correction should not duplicate an effect.

4

Move to the intended session policy.

Kit: compare owner movement with Hello Sim's authoritative tick. Replay: use Session for the direct server/client lifecycle, Party → Room for invitations and migration, or the admitted P2P runner for deterministic peers.

Owner motion and replicated state

```
# Build the existing co-op example.
nix develop --command bash build/build_scripts.sh examples/hello_net

# Terminal 1: launch, then choose Host.
nix develop --command bash -c '"$GODOT" --path examples/hello_net'

# Terminal 2: launch, then choose Join.
nix develop --command bash -c 'HELLO_LATENCY=120 "$GODOT" --path examples/hello_net'
```

Read: examples/hello_net/scripts/player.odin · docs/kit/quickstart.md

Commands run from the repository root. The development shell supplies the pinned toolchain and `GODOT`. These are existing project commands, not a new build pipeline.

Once the pickup works, add one dependency at a time: a cooldown that gates movement, a projectile with a lifetime, a moving door. Each addition should answer the same question: if this tick changes, which values must be rebuilt with it?

KEEP BESIDE THE CODE

Diagrams and reading.

The diagrams are standalone SVG files. The reading library brings together the tutorials, architecture notes, and examples behind this guide. Each page includes its original source as a download.

01	The two authoring models	SVG ↓
02	The engine boundary	SVG ↓
03	A Kit command, end to end	SVG ↓
04	Inside and outside a frame	SVG ↓
05	Replay's authoritative network path	SVG ↓
06	Restore and replay a suffix	SVG ↓
↳	Desk reference: rules, paths, commands	MD ↓
↳	Print edition of the guide	PDF ↓

Built from repository 446540144e3c with local source changes, 2026-10-30. Source snapshots are included. The browser models illustrate behavior; they do not execute the Odin frameworks. Replay remains experimental.